



AI with Rav

Learn AI the way you actually think.



DAY 10 of 30

MACHINE LEARNING

K-Nearest Neighbours — you are your closest friends

WHAT YOU'LL LEARN TODAY

- › The "5 closest friends" idea behind KNN
- › What "distance" really means (a simple ruler)
- › How the neighbours VOTE to decide
- › How to pick K — and why it matters
- › Where KNN quietly breaks



K-Nearest Neighbours — you are your closest friends

In One Line: To judge something NEW, KNN looks at the few things it sits closest to, and copies them. "Show me your friends, and I'll tell you who you are." That's the whole algorithm.

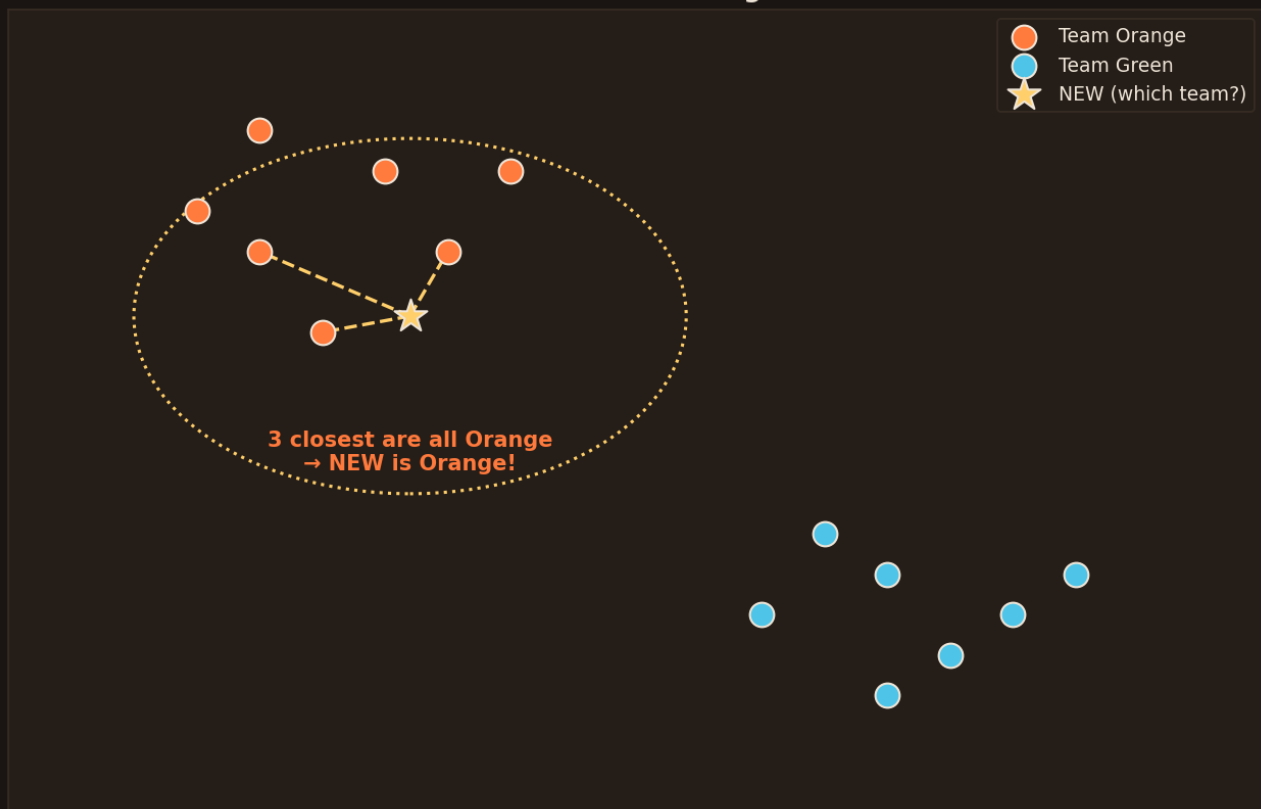
Start here: the new student in class

A new student joins your class. You don't know anything about them yet. But you notice **who they sit with** — the back-bench cricket gang, all day. Even before they say a word, you guess: *"This one's a cricket kid too."*

You just did **KNN** in your head. There's an old line: **"You are the average of the 5 people you spend the most time with."** KNN is exactly that idea, turned into a machine. To judge anything new, it looks at its **closest neighbours** and copies them. It might be the simplest idea in all of ML — and it just works.

The whole idea in one picture

Which team does the NEW point belong to?
Look at its 3 closest neighbours.



A new point (yellow star) appears. Look at its 3 closest neighbours. All 3 are Orange — so the new point is called Orange too.



Say we already know two groups — **Team Orange** and **Team Green** (maybe oranges vs green apples, or safe vs fraud customers). Now a **new** thing shows up (the yellow star) and we don't know its team.

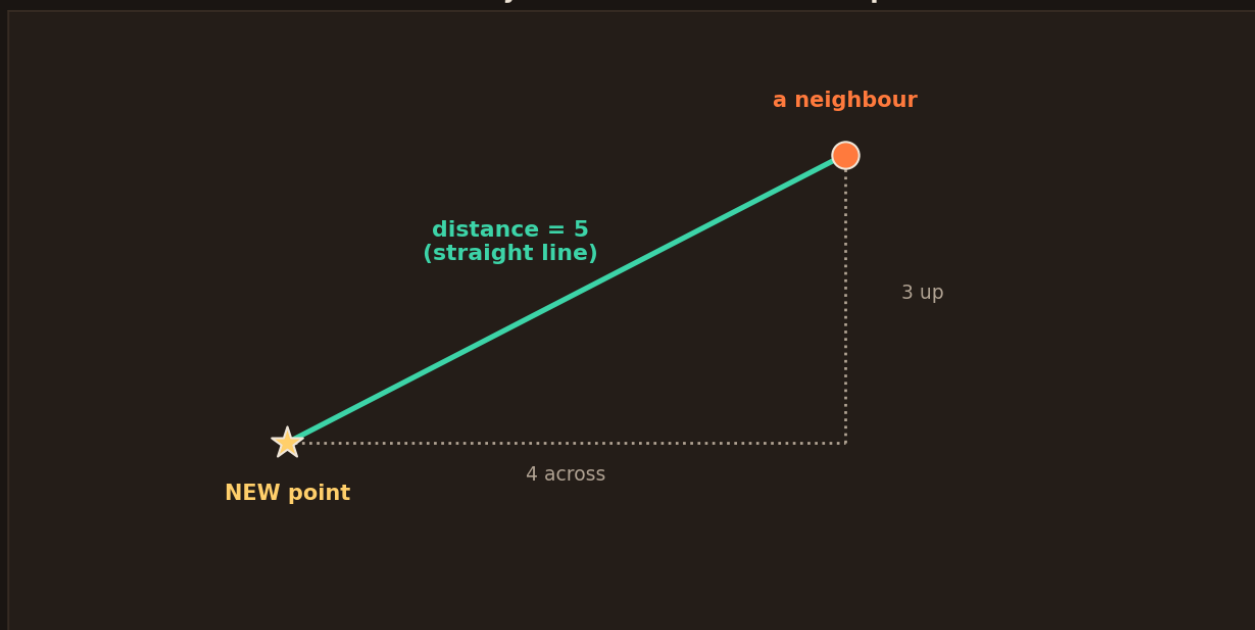
- Find its **closest neighbours** — the points sitting nearest to it.
- Look at what **team** those neighbours belong to.
- Give the new point the **same team** as most of its neighbours.

In the picture, the star's 3 closest neighbours are **all Orange**. So KNN says: "You're sitting with the Orange crowd — you're Orange too." No training, no maths, no fancy line. Just: *who are you sitting next to?*

What "distance" really means

To find the "closest" neighbours, the machine needs to measure **how far apart** two points are. And "distance" here is nothing scary — it's just a **ruler** between two dots.

"Distance" is just a ruler between two points



Distance is just a straight-line ruler. Go 4 steps across and 3 steps up — the straight line between them is 5. That's the distance.

Remember school geometry? Go **4 steps across** and **3 steps up**, and the straight line joining start to end is **5**. That "straight-line ruler" is all KNN uses.



Distance = $\sqrt{(\text{difference across})^2 + (\text{difference up})^2}$ — for our picture: $\sqrt{(4^2 + 3^2)} = \sqrt{(16 + 9)} = \sqrt{25} = 5$. It's just the length of the straight line between two points. The "across" and "up" are simply the differences in each feature (say, sweetness and size of a fruit). Smaller distance = more alike. That's the only maths in KNN.

The neighbours VOTE

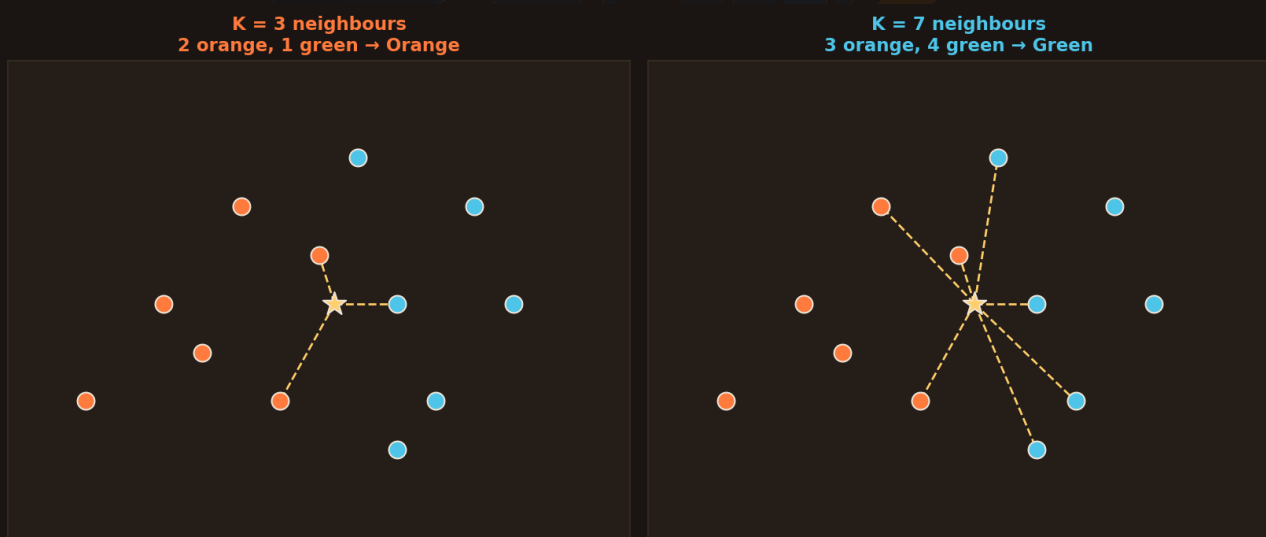
KNN doesn't just look at ONE closest neighbour — it looks at the **K closest** (K is just a number you pick, like 3 or 5). Then those K neighbours **vote**, exactly like Random Forest's trees voted.

- Pick **K** = how many neighbours get to vote (say K = 5).
- Find the **5 closest** points to the new one.
- Whichever team has **more votes** among those 5 → that's the answer.

If 4 of the 5 closest neighbours are "fraud" and 1 is "safe", KNN says **fraud** — the majority wins. It's the same "wisdom of the neighbours" as your new student sitting with the cricket gang: most of the people around them point one way, so that's your best guess.

Choosing K — the one setting that matters

Here's the fun part: **the number K can change the answer**. Look what happens to the SAME new point when we let more neighbours vote:



Same new point, sitting on the border. With K=3 the nearest are mostly Orange → Orange. With K=7 we reach further out and pick up more Green → Green. K changes the answer!

- **K too small (like K=1)** → it listens to just ONE neighbour. If that one is a weird oddball (noise), you get fooled. Too jumpy.



- **K too big** → it asks so many neighbours that it starts including far-away points that aren't really similar. The local flavour gets washed out.
- **K just right** (often a small odd number like 5 or 7) → enough votes to cancel out oddballs, but still "local".

Tip: pick an **odd** K (3, 5, 7...) for yes/no problems — an odd number can't tie, so there's always a clear winner. And try a few values of K, keep the one that predicts new data best. This "picking K" is the main skill in KNN.

There's no "training" — and that's the twist

Every algorithm so far (Linear, Logistic, Trees, Forest, XGBoost) **learned** something first, then predicted fast. KNN flips it:

- **Training** → basically nothing. KNN just **remembers all the data**. That's it.
- **Predicting** → this is where the work happens: for each new point it measures distance to **every** stored point to find the neighbours.

That's why KNN is called a **"lazy" learner** — it's lazy at training (just memorises), but hard-working at prediction time (measures everything). The opposite of the others, which work hard once and then predict instantly.

Try it — a few lines

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5) # K = 5 voters
model.fit(X_train, y_train)                # "fit" = just remember the data

print(model.predict([[sweetness, size]]))  # -> which team the neighbours vote for
```

``n_neighbors=5`` is your **K**. Notice ``fit`` here doesn't really "learn" — it just stores the points. All the real work happens inside ``predict``, when it hunts for the 5 nearest neighbours and counts their votes.

Where KNN quietly breaks

KNN is simple and lovely — but it has real weak spots an engineer must know:



- **Slow on big data** → to predict ONE point it measures distance to EVERY stored point. A million rows = a million measurements each time. Painfully slow.
- **Features must be on the same scale** → if "salary" is in lakhs and "age" is in years, salary's big numbers drown out age in the distance. Always **scale** features first (put them on a common ruler), or distance is meaningless.
- **Struggles with too many features** → in very high dimensions, everything ends up looking "equally far" and "closest" stops meaning much (the famous *curse of dimensionality*).
- **Sensitive to K and to noise** → a bad K, or messy outliers sitting nearby, can flip the vote.

The golden rule: **always scale your features before KNN**. Distance is the whole engine — if one feature's numbers are huge, it hijacks the ruler and KNN listens only to that feature. Put everything on the same scale first.

Recap — the 20-second version

- KNN = **"you are your closest neighbours."** Judge new things by what sits near them.
- **Distance** is just a straight-line ruler; smaller distance = more alike.
- The **K nearest** neighbours **vote**; the majority team wins.
- **K** is the key setting — small = jumpy, big = blurry; pick an odd number and test.
- It's **lazy**: no real training, just remembers data — but **scale your features first**, always.

Next up — Video 11: Support Vector Machines (SVM). Instead of asking neighbours, what if we drew the single BEST dividing line between two groups — the one with the widest safety gap? The algorithm that finds the perfect boundary. See you Day 11.

